

セキュリティ説明

SocraMetry はエラーログを貼り付けて使う製品です。エラーログには API キーや接続文字列が日常的に混入するため、「**混入する前提**」で**システム側が守る設計**にしています。本書は、何をしていて、何を保証していないかをまとめたものです。

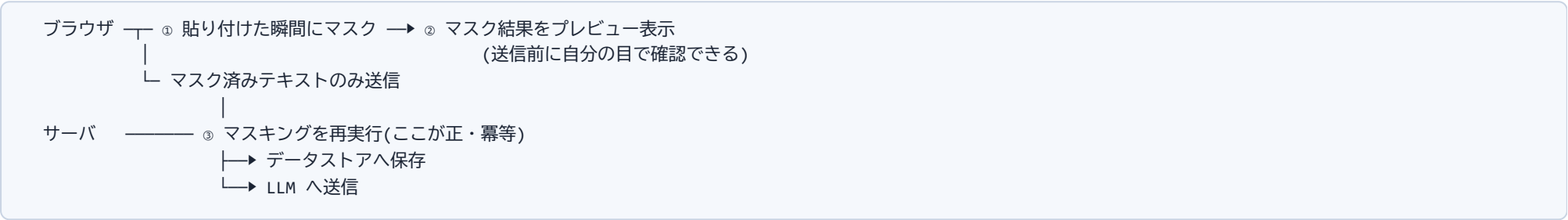
1. この製品固有のリスクを直視する

#	リスク	なぜ避けられないか
R1	入力に認証情報が混入する	貼り付けるのはエラーログ。ログには API キー・トークン・接続文字列が日常的に含まれる
R2	入力が外部の LLM プロバイダに送られる	診断のためにエラーテキストを LLM に渡す必要がある

R1 は「ユーザーが気をつける」では解決しません。本人が気づかずに貼り付けるから混入するのであり、**システム側で除去する以外に方法がない**——これが設計の出発点です。

2. 秘匿情報の自動マスキング(R1 への対策)

データストアに保存する前・LLM に送る前に、認証情報を自動でマスキします。クライアントとサーバの**二段構成**で、同じマスキング関数を両側で実行します。



クライアント側のマスクは UX のためであり、保証はサーバ側の再実行が担います。同一の関数を両側で使うため、**プレビューで見た内容と実際に送られる内容がずれません**。生のテキストはブラウザから外に出ません。

自動で検出・マスクするもの

対象	例・検出方法	置換後
API キー類	sk-, ghp_, AKIA, AIza 等のプレフィックス + 長さ	[REDACTED_KEY]

Bearer トークン	Authorization: Bearer ...	[REDACTED_TOKEN]
接続文字列	postgres://user:pass@... の資格情報部	[REDACTED_CREDENTIALS]
JWT	eyJ... で始まる 3 セグメント	[REDACTED_JWT]
メールアドレス	正規表現	[REDACTED_EMAIL]
ローカル絶対パス	ファイル名より前をすべて伏せる(中間ディレクトリの顧客名・プロジェクト名も残さない)	<path>/<file>
登録済みの社名・製品名	管理側で登録した語の完全一致	[REDACTED_NAME]

正直にお伝えします — 自動では消せないものがあります。

未登録の顧客名などの固有名詞は、正規表現では機械的に検出できません。そのため**マスク結果を送信前にプレビューで表示**し、利用者自身が確認・修正できる導線にしています。「マスクしているから安全」ではなく、何をしていた何を保証していないかを明示する方針です。

3. LLM プロバイダへのデータの扱い(R2 への対策)

項目	方針
学習への利用	ゲートウェイ(OrcaRouter)側で入力を学習に使わせない設定を確認し、明記する
送信内容	マスク済みエラーテキスト・コード断片のみ。メールアドレスや氏名は送らない
利用者への説明	外部 LLM に送られること・マスクの内容と限界を UI に明記。送信内容は事前にプレビューで確認できる

4. API キーと認証の守り方

API キーをブラウザに出さない

✕ してはいけない構成 ブラウザ → [APIキー] → LLM ゲートウェイ
↑ DevTools で誰でも見える

✓ SocraMetry ブラウザ → サーバ関数 → [APIキー] → LLM ゲートウェイ
↑ 実行環境の環境変数にのみ存在

キーはクラウド実行環境の環境変数にのみ置き、リポジトリにも CI にも置きません。CI が知るシークレットをデプロイ用の 1 つに絞ることで、漏洩経路そのものを減らしています。

認証・アクセス制御(v0.1)

項目	実装
登録	招待コード必須 。公開 URL に無制限のサインアップを置かない(コスト流出防止を兼ねる)
パスワード	ソルト付き scrypt でハッシュ化して保存(平文保存しない)
セッション	JWT を HttpOnly / Secure / SameSite=Lax の Cookie に格納。有効期限 24 時間
他人のデータ	データストアのキーに所有者 ID を含める構造のため、 他人のセッション ID を渡しても「存在しない」としか返らない
入力検証	サーバ側で必ず検証(文字数上限・許可リスト・ID 形式)。フロントの検証は UX のためで、防御はサーバが担う

5. ログに個人情報・秘匿情報を残さない

記録する	記録しない
セッション ID(個人を特定しない)・役割・モデル名・トークン数・レイテンシ・推定コスト・エラーコード	エラーテキストの本文・コード断片 ・メールアドレス・氏名・パスワード・例外に含まれる入力値

ログの目的は運用とコスト把握であり、入力内容の再現ではありません。本番環境では入力の「長さ」だけを記録し、本文は残しません。利用者は自分のセッションデータを削除できます。

6. 濫用への防御

層	実装
レート制限	セッション作成 10 回/時/ユーザー。LLM コストの流出を防ぐ
トークン上限	1 セッション 80,000 トークンで打ち切り。暴走的な生成を止める
冪等性	同じ設問への二重回答は同じ結果を返す。API を直接叩かれても LLM を連打させられない
XSS	LLM の出力は信頼できない入力として扱い、HTML として描画しない
通信	すべて HTTPS(実行環境が TLS を終端)

7. v0.1 に対応していないこと

やっていないことを「やった」とは書きません。以下は v0.2 以降の対応です。

項目	現状の扱い
マルチテナント・ロール管理 / 監査ログ	v0.2 で対応。v0.1 は招待制の限定公開で運用
SSO / OIDC	v0.2 で対応。v0.1 は簡易ログインで代替
パスワードリセット	v0.2 で対応。招待コード制の限定公開のため
WAF / DDoS 対策	実行環境側に委ねる